1. Algorithm Analysis

- **Big-O** (O): Upper Bound. **Omega** (Ω): Lower Bound. **Theta** (Θ): Tight Bound.

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < 2^{2n}$
$\log_a n < n^a < a^n < n! < n^n$
Notable: $\sqrt{n} \log n = O(n)$; $O(\log(n!)) = O(n \log n)$; $O(2^{2n}) \neq O(2^n)$

Master Theorem

Shortcut for recurrences: $\mathbf{T(N) = aT(N/b) + O(N^d)}$
Where $a \geq 1$ (subproblems), $b > 1$ (factor size is reduced), and $O(N^d)$ is work done outside recursive calls. Compare $\mathbf{a}$ and $\mathbf{b^d}$:

- $a < b^d \implies \mathbf{O(N^d)}$ (Work dominated by root)
- $a = b^d \implies \mathbf{O(N^d \log N)}$ (Work evenly distributed)
- $a > b^d \implies \mathbf{O(N^{\log_b a})}$ (Work dominated by leaves)

2. Sorting Algorithms

Algorithm	Best Time	Worst Time	Space
Bubble Sort	$\Omega(N)$	$O(N^2)$	$O(1)$
Insertion Sort	$\Omega(N)$	$O(N^2)$	$O(1)$
Selection Sort	$\Omega(N^2)$	$O(N^2)$	$O(1)$
Mergesort	$\Omega(N \log N)$	$O(N \log N)$	$O(N)$
Quicksort	$\Omega(N \log N)$	$O(N^2)$	$O(\log N)$

Mergesort (Divide & Conquer)

Stable sort. $T(N) = 2T(N/2) + O(N) \implies O(N \log N)$.

Quicksort & Quickselect

Pick a pivot, partition into $<$, $=$, $>$. Random pivot avoids $O(N^2)$ worst-case. In-place, unstable. **Quickselect**: Finds k-th smallest element. Expected time $O(N)$.

3. Binary Heaps (Priority Queue)

Complete binary tree. **Max-Heap**: Parent $\geq$ Children.

- **Array Layout**: Root at 0. Left: $2i + 1$. Right: $2i + 2$. Parent: $(i - 1)/2$.
- **Insert**: Append end, `bubbleUp()`. $O(\log N)$.
- **ExtractMax**: Swap root & last, pop, `bubbleDown()`. $O(\log N)$.
- **BuildHeap**: `bubbleDown()` from $N/2$ down to 0. Time: $\mathbf{O(N)}$.

4. BSTs & Balanced Trees

BST: Search, Insert, Delete run in $O(h)$. Worst case $O(N)$. Balanced trees (AVL, Treaps) enforce $h = O(\log N)$.

AVL Trees (Self-Balancing)

Rotations: **LL**: Right rot. **RR**: Left rot. **LR**: Left rot left child, Right rot root. $O(1)$ for insert, $O(\log n)$ for deletion.

Tree Augmentation & Intervals

Store `size` of subtree at each node to find rank (k-th smallest) in $O(\log N)$.

5. Graph Fundamentals

Let V = Vertices (N), E = Edges (M).

- **Adjacency Matrix**: $O(V^2)$ space. $O(1)$ query edge.
- **Adjacency List**: $O(V + E)$ space. $O(deg(v))$ to iterate neighbors.
- **Edge List**: Array of all edges '(u, v, w)'. $O(E)$ space.

Algorithm	Runtime	Primary Use Case
BFS / DFS	$O(V + E)$	Traversal, Connected Comps
Toposort	$O(V + E)$	Ordering DAG dependencies
Dijkstra (PQ)	$O((V + E) \log V)$	SSSP (No negative edges)
Bellman-Ford	$O(V \cdot E)$	SSSP (Handles -ve edges)
DAG SSSP	$O(V + E)$	SSSP on DAGs only
Kruskal	$O(E \log E)$	MST (Sparse graphs)
Prim	$O((V + E) \log V)$	MST (Dense graphs)

6. Graph Traversal

Breadth-First Search (BFS)

Shortest paths on **unweighted** graphs.

```
# Time: O(V+E)
q = [start]
vis[start] = True

while q:
  u = q.pop(0) # Process node
  for v in adj[u]:
    if not vis[v]:
      vis[v] = True; q.append(v)
```

Depth-First Search (DFS) & Toposort

Uses recursion (stack). **Topological Sort**: Valid DAG order. Add vertex to front of list *after* visiting all neighbors (post-order).

```
# Time: O(V+E)
def dfs(u):
  vis[u] = True
  for v in adj[u]:
    if not vis[v]: dfs(v)
  topo.append(u) # Toposort: Reverse list afterwards!
```

Advanced BFS

- **Multi-Source BFS**: Shortest path from *any* node in a set. Set 'dist=0' and push *all* sources to queue initially.
- **0/1 BFS**: Edges of weight 0 or 1. Use a **Deque**. If edge weight is 0, 'push-front'. If weight 1, 'push-back'. $O(V + E)$.

7. Union-Find Disjoint Sets (UFDS)

Superfast $O(\alpha(N))$ ops via **Path Compression** and **Union by Rank**.

```
def find(i): # Path Compression
  if parent[i] == i: return i
  parent[i] = find(parent[i])
  return parent[i]

def union(i, j): # Union by Rank
  rootI, rootJ = find(i), find(j)
  if rootI != rootJ:
    if rank[rootI] < rank[rootJ]: parent[rootI] = rootJ
    elif rank[rootI] > rank[rootJ]: parent[rootJ] = rootI
    else: parent[rootJ] = rootI; rank[rootI] += 1
```

8. Minimum Spanning Trees (MST)

Kruskal's Algorithm

Greedy: Sort an **Edge List** by weight ascending. Add edges if they don't form cycles (use UFDS). Best for sparse graphs.

```
# Time: O(E log E) to sort edges.
# UFDS takes amortized O(1). Loop runs E times. Total: O(E log E).
```

Integer weights in $[0, c)$: use c buckets (bucket queue); find-min in $O(c) \Rightarrow$ total $O(n(c + m))$.

Prim's Algorithm

Greedy: Grow from start node. Use Min-Heap PQ to pick cheapest edge connecting tree to an unvisited node. Best for dense graphs.

```
# Time: O((V+E) log V)
pq.push((0, start_node)) # (weight, node)

while pq:
  w, u = pq.pop()
  if inMST[u]: continue # Lazy deletion
  inMST[u] = True
  mstCost += w

  for e in adj[u]:
    if not inMST[e.to]:
      pq.push((e.w, e.to))
```

9. Single-Source Shortest Path (SSSP)

Dijkstra's Algorithm

Greedy Min-Heap. Cannot handle negative weight edges.

```
# Time: O((V+E) log V)
dist[start] = 0
pq.push((0, start)) # (dist, node)

while pq:
  d, u = pq.pop()
  if d > dist[u]: continue # Lazy deletion / Stale
  for e in adj[u]:
    if dist[u] + e.w < dist[e.to]:
      dist[e.to] = dist[u] + e.w # Relax edge
      pq.push((dist[e.to], e.to))
```

Bellman-Ford Algorithm

Handles **negative edges**. Iterate through an **Edge List**, relaxing all E edges $V - 1$ times.

```
# Time: O(V * E)
dist[start] = 0

for _ in range(V - 1):
  relaxed = False
  for e in edgeList: # e.u to e.v with weight e.w
    if dist[e.u] + e.w < dist[e.v]:
      dist[e.v] = dist[e.u] + e.w
      relaxed = True
  if not relaxed: break # Optimization: early exit
# Detect negative cycles: Run one more (V-th) time.
# If any edge relaxes, a negative cycle exists.
```

DAG Shortest Paths

If graph is a DAG, **Toposort** the nodes. Relax outgoing edges in toposort order. Guarantees shortest path in $\mathbf{O(V+E)}$.

10. Dynamic Programming (DP)

LIS — $O(n^2)$

`dp[i]` = length of LIS ending at index i.

$$\texttt{dp[i]} = 1 + \max_{\substack{j<i \\ a[j]<a[i]}} \texttt{dp[j]}, \quad \texttt{dp[0]} = 1$$

Answer: $\max_i$ `dp[i]`.

Unbounded Knapsack — $O(nC)$

Items reusable.

$$\texttt{dp[i][c]} = \max\big(\texttt{dp[i-1][c]},\ \texttt{dp[i][c-w]} + v\big)$$

Iterate c *forward* for 1D table.

Bounded Knapsack — $O(nC)$

Each item i has `cnt[i]` copies. Use sliding-window max (Max Queue) per residue lane `c % w`; window size `cnt[i]+1`. See Max Queue section.

Edit Distance — $O(nm)$

`dp[i][j]` = min edits to transform `a[0..i-1]` to `b[0..j-1]`.

$$\texttt{dp[i][j]} = \begin{cases} \texttt{dp[i-1][j-1]} & \text{if } a[i] = b[j] \\ 1 + \min\big(\texttt{dp[i-1][j]},\ \texttt{dp[i][j-1]},\ \texttt{dp[i-1][j-1]}\big) & \text{else} \end{cases}$$

Base cases: `dp[i][0] = i`, `dp[0][j] = j`.

LCS — $O(nm)$

`dp[i][j]` = length of LCS of `a[0..i-1]` and `b[0..j-1]`.

$$\texttt{dp[i][j]} = \begin{cases} \texttt{dp[i-1][j-1]+1} & a[i] = b[j] \\ \max\big(\texttt{dp[i-1][j]},\ \texttt{dp[i][j-1]}\big) & \text{else} \end{cases}$$

Recovery

Follow child pointers from `dp[n][m]`; append only on diagonal moves (matched characters).

Vertex Cover on Tree — $O(n)$

Post-order DFS; `dp[v][0/1]` = min cover of subtree of v, not including / including v.

$$\texttt{dp[v][1]} = 1 + \textstyle\sum_c \min\big(\texttt{dp[c][0]},\ \texttt{dp[c][1]}\big)$$
$$\texttt{dp[v][0]} = \textstyle\sum_c \texttt{dp[c][1]}$$

Answer: $\min\big(\texttt{dp[root][0]},\ \texttt{dp[root][1]}\big)$.

DP Pattern Reference

Pattern	Template
Take or skip	$\max(\text{skip}_i,\ \text{take}_i)$
Partition at k	$\min_{k<i}(\texttt{dp[k]} + \text{cost}(k,i))$
Two sequences	2D table; iterate both indices
Tree DP	post-order DFS; combine children at v
Interval DP	`dp[l][r]`; base case `dp[i][i]`

- **1. Reformulate recurrence:** Reduce branching so each state depends on $O(1)$ prior states rather than scanning $O(n)$ predecessors. *Example:* Unbounded knapsack naive: loop over all copy counts $j \Rightarrow O(nC^2)$. Reformulation: `dp[n][c] = max(dp[n-1][c], dp[n][c-w]+v)` (take exactly 1 more or none) $\Rightarrow O(nC)$.
- **2. Sliding-window max (monotone deque):** Recurrence `dp[i] = max(dp[j]) + f(i)` over a window $[i-k, i-1]$: maintain a deque of candidate indices. Remove from front when out of window; remove from back when new value dominates. Each index enters/leaves deque once $\Rightarrow$ amortised $O(1)$ per state.
- **3. Prefix sums:** If recurrence sums over a contiguous range of prior values, precompute prefix sums. Reduces per-state cost from $O(n) \to O(1)$.
- **4. Iteration direction (1D table reuse):** *Unbounded knapsack:* iterate capacity *forward* (allows reusing same item in same row). *0/1 knapsack:* iterate capacity *backward* (prevents using same item twice). Enables rolling-array space optimisation to $O(C)$.
- **5. Matrix exponentiation:** Any fixed-width linear recurrence (e.g. Fibonacci) can be computed in $O(k^3 \log n)$ using repeated matrix squaring.
- **6. Divide-and-conquer optimisation:** When the optimal split-point $\text{opt}(i,j)$ is monotone in i (quadrangle inequality), reduce $O(n^2)$ partition DP to $O(n \log n)$.

11. Advanced Graph Modeling

Time-Expanded Graphs (The Meeting Problem)

When state depends on time, vertices become tuples: **(Node, Time)**.

- **Wait Edges:** Connect $(P_u, T_1) \to (P_u, T_2)$ with weight 0.
- **Meeting Edges:** Connect $(P_u, T) \leftrightarrow (P_v, T)$ with weight 0.

The Expressway Problem (Graph Layering)

If you are allowed to make K edges "free" on a shortest path, create $K+1$ copies of the graph. Normal edges connect $(u,k) \to (v,k)$ with weight w. Free edges connect $(u,k) \to (v,k+1)$ with weight 0. Target is $\min(\text{dist}[t][0 \ldots K])$. Time: $O(K(V+E)\log V)$.

Counting Triangles (Matrix Multiplication)

Given Adjacency Matrix A. Compute A^3. The diagonal entry $A^3[i][i]$ is paths of length 3 from i to i. Total Triangles = $\text{Trace}(A^3)/6$. Time: $O(V^{2.807})$.

12. Master Data Structures Table

Data Structure	Search	Insert	Delete	Max/Min
Sorted Array	$O(\log N)$	$O(N)$	$O(N)$	$O(1)$
Hash Table	$O(1)$ exp	$O(1)$ exp	$O(1)$ exp	$O(N)$
AVL Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$
Binary Heap	$O(N)$	$O(\log N)$	$O(\log N)$**	$O(1)$
UFDS	$O(\alpha(N))$	N/A	N/A	N/A

**$O(\log N)$ to extract-min/max, arbitrary delete is $O(N)$.

13. Max-Queue (Special Trick)

Enqueue, Dequeue, Get-Max, Add-To-All in $\mathbf{O(1)}$ amortized time. Implement with `in_stack` and `out_stack`. Each entry stores `(val, max_so_far)`.

- **Max-stack**: On push, store `(v, max(v, stk.top.max))`. `peek_max` returns `stk.top.max` in $O(1)$.
- **Enqueue**: Push `val - offset` onto `in_stk`.
- **Dequeue**: If `out_stk` is empty: pop all entries from `in_stk` onto `out_stk` one by one, recomputing `max_so_far` as they arrive (this reverses their order). Then pop the top of `out_stk`.
- **Get-Max**: `offset + max(in_stk.max, out_stk.max)`.
- **Add-To-All**: Increment global `offset` variable by the given value. $O(1)$. When enqueuing, storing `val - offset` maintains the invariant `actual = stored + offset`.

Usage in Bounded Knapsack

Allows solving Bounded Knapsack in $O(N \cdot W)$ instead of $O(N \cdot W \cdot C)$. We optimize the transition over multiple copies of the same item using a Max-Queue on the residue classes modulo weight w.

```
for w, v, cnt in items:
  for k in range(w): # residue class
    max_q = MaxQueue()
    for m in range(...): # 0, 1, 2, ...
      c = m*w + k
      max_q.enqueue(dp[c] - m*v)
      if max_q.size > cnt + 1: max_q.dequeue()
      dp[c] = max_q.get_max() + m*v
```